Python Code

```python
import heapq
import matplotlib.pyplot as plt
import numpy as np
import time

#-----------------------------------------------------------
# Global Vars
explored_nodes = []
open_cost = {}
open_list = []
closed_list = set()

#-----------------------------------------------------------
# Different Action Primitives
def move(child_t, c2c, step_size, delta_theta):
    # print(f"CHOLD_T: {child_t}")
    ori = child_t[2]
    new_ori = ori + delta_theta
    theta = np.deg2rad(new_ori)

    new_x = child_t[0] + step_size * np.cos(theta)
    new_y = child_t[1] + step_size * np.sin(theta)

    new_state = (new_x, new_y, new_ori)
    new_cost = c2c + step_size

    return new_state, new_cost


def move_fwd(child_t, c2c, step_size):
    return move(child_t, c2c, step_size, 0)

def move_fwd_ccw30(child_t, c2c, step_size):
    return move(child_t, c2c, step_size, 30)

def move_fwd_ccw60(child_t, c2c, step_size):
    return move(child_t, c2c, step_size, 60)

def move_fwd_cw30(child_t, c2c, step_size):
    return move(child_t, c2c, step_size, -30)

def move_fwd_cw60(child_t, c2c, step_size):
    return move(child_t, c2c, step_size, -60)

#-----------------------------------------------------------
def discretize(node):

    x, y, theta = node

    ix = int(np.floor(x / 0.5 + 0.5))   # nearest 0.5 bin
```

```python
    iy = int(np.floor(y / 0.5 + 0.5))
    itheta = int((theta % 360) // 30)   # 0.11

    return (ix, iy, itheta)


#-----------------------------------------------------------
def run_AStar(start_pos, goal_pos, step_size, all_collisions, clearance):
    """
    Initialize the alg by adding the start state to the open list and then
    calling the function to compare against the goal state. A* will continue
    until the goal is reached, at which point the path to the goal will be generated.
    """
    global t_init
    global open_list

    t_init = time.perf_counter()



    plt.plot(start_pos[0], start_pos[1], 'o', color="orange", label = "start", zorder = 10)
    plt.plot(goal_pos[0], goal_pos[1], '*', color="lime", label = "goal", zorder = 10)
    plt.legend(loc="upper left", bbox_to_anchor=(0.8, 1.5))

    # initialize the parent dictionary with the start state (which has no parent)
    child_t = tuple(goal_pos)

    # discretize child_t
    child_t_disc = discretize(child_t)
    info = (child_t_disc, child_t)

    # initialize cost to come and total cost
    c2c = 0
    ct = c2c + np.sqrt((start_pos[0]-goal_pos[0])**2 + (start_pos[1]-goal_pos[1])**2)

    # add to dictionary
    open_cost[child_t_disc] = (None, c2c, goal_pos)

    # add to heap
    heapq.heappush(open_list, (ct, info))

    # start loop
    t_fin = compare_against_goal(goal_pos, start_pos, step_size, all_collisions, clearance)

    return t_init, t_fin

#-----------------------------------------------------------
def compare_against_goal(goal_pos, start_pos, step_size, all_collisions, clearance):
    """
    The loop that runs until the open list is empty
    """
    while open_list:
```

```python
        # remove the cheapest cost entiry from opon_list
        ct, (child_t_disc, child_t) = heapq.heappop(open_list)
        # print(f"CHILD T DISC: {child_t_disc}, CHILD T: {child_t}")

        # extract info from dictionary
        parent_disc_state, c2c, actual_state = open_cost[child_t_disc]
        # actual_state = tuple(map(int, actual_state))

        # print(f"PARENT DISC STATE: {parent_disc_state}, ACTUAL STATE: {actual_state}")

        # Check to see if the start_pos has been reached
        if (actual_state[0] - start_pos[0])**2 + (actual_state[1] - start_pos[1])**2 <= 1.5**2:
            print(f"GOAL REACHED; COST IS {c2c}")
            t_fin = time.perf_counter()
            order = generate_path(goal_pos, start_pos, open_cost.get(child_t_disc), clearance)
            animate_search_and_path(order, explored_nodes, t_fin)
            return t_fin

        # add the child to the closed list if it's not already there
        # and generate the next movements
        if child_t_disc not in closed_list:
            closed_list.add(child_t_disc)
            explored_nodes.append(child_t)

            generate_possible_moves(start_pos, parent_disc_state, c2c, actual_state, step_size, all_collisions)

    # return t_fin
#-------------------------------------------------------------
def generate_possible_moves(start_pos, parent_disc_state, c2c, actual_state, step_size, all_collisions):
    """
    From the previous location, generate possible next steps,
    evaluate to ensure they aren't in a collision or out of
    bounds, and then, if the entry isn't in open_cost or
    the cost is less than the entry in open_cost, add that entry
    to open_list
    """
    global open_list

    # define action obtions
    Actions_Set = (
        move_fwd,
        move_fwd_ccw30,
        move_fwd_ccw60,
        move_fwd_cw30,
        move_fwd_cw60
    )

    # run through all possible actions
    for action in Actions_Set:
```

```python
        # generate new position after action
        new_state, new_cost = action(actual_state, c2c, step_size)
        x, y, theta = new_state
        # print(f"ACTUAL STATE: {actual_state}")
        # print(f"NEW SSTATE: {new_state}")


        # discretize new position
        new_state_disc = discretize(new_state)
        x_d, y_d, theta_d = new_state_disc

        # update cost
        ct = new_cost + np.sqrt((x - start_pos[0])**2 + (y - start_pos[1])**2)

        # ct = new_cost + np.sqrt(
        #     (start_pos[0] - actual_state[0])**2 +
        #     (start_pos[1] - actual_state[1])**2 +
        #     theta_equiv_dist(start_pos[2], actual_state[2], step_size)**2
        # )

        # check boundaries
        if x_d < 0 or x_d >= all_collisions.shape[1] or y_d < 0 or y_d >= all_collisions.shape[0]:
            continue

        if all_collisions[y_d, x_d]:
            continue

        # check collisions
        if all_collisions[y_d, x_d]:
            continue

        # add to open list if cost is less than previous entry or if it isnt already in the open list
        if new_state_disc not in open_cost or open_cost.get(new_state_disc)[1] > new_cost:
            open_cost[new_state_disc] = (actual_state, new_cost, new_state)
            heapq.heappush(open_list, (ct, (new_state_disc, new_state)))

#-----------------------------------------------------------
def theta_equiv_dist(theta1, theta2, step_size):
    d = abs((theta1 - theta2) % 360)
    d = min(d, 360 - d)        # wrap to shortest angle
    return (d / 30.0) * step_size

#-----------------------------------------------------------
def generate_path(goal_pos, start_pos, final_loc, clearance):
    parent, cost, child = final_loc

    order = [child]

    while parent is not None:
        order.append(parent)
        parent, cost, child = open_cost[discretize(parent)]
```

```python

    draw_obstacle_course(order, clearance)
    return order[::-1]

#-----------------------------------------------------------
def animate_search_and_path(order, explored_nodes, t_fin):
    """
    Animate the search and final path
    """
    global t_init
    plt.ioff()

    # --- Timing ---
    dt = t_fin - t_init
    print(f"The algorithm takes {dt:.3f} s to run")
    print(f"EXPLORED NODES: {len(explored_nodes)}")

    # --- Animation tuning ---
    num_updates = 75
    growth_rate = max(1, len(explored_nodes) // num_updates)

    base_pause = 0.001
    pause_time = min(0.05, base_pause * growth_rate)

    # --- Batch storage for quiver ---
    qx, qy, qu, qv = [], [], [], []



    # # --- Mark start and goal from final path ---
    # if order:
    #     plt.plot(order[-1][0], order[-1][1], 'go', markersize=6)  # start
    #     plt.plot(order[0][0],  order[0][1],  'mo', markersize=6)  # goal

    # --- Animate search tree ---
    for k, node in enumerate(explored_nodes):
        disc_node = discretize(node)

        if disc_node in open_cost:
            parent, _, _ = open_cost[disc_node]

            if parent is not None:
                parent_disc = discretize(parent)

                if parent_disc in open_cost:
                    parent_state = open_cost[parent_disc][2]

                    qx.append(parent_state[0])
                    qy.append(parent_state[1])
                    qu.append(node[0] - parent_state[0])
                    qv.append(node[1] - parent_state[1])

        if k % growth_rate == 0 or k == len(explored_nodes) - 1:
            if qx:  # only draw if there's data
```

```python
                plt.quiver(qx, qy, qu, qv,
                           angles='xy', scale_units='xy', scale=1,
                           color='b', width=0.002)

            plt.draw()
            plt.pause(pause_time)

            qx.clear()
            qy.clear()
            qu.clear()
            qv.clear()

    # --- Animate final path ---
    order.reverse()
    for i in range(1, len(order)):
        x0, y0 = order[i-1][0], order[i-1][1]
        x1, y1 = order[i][0], order[i][1]

        dx = x1 - x0
        dy = y1 - y0

        plt.quiver(x0, y0,
                   dx, dy,
                   angles='xy', scale_units='xy', scale=1,
                   color='r', width=0.004)

        plt.draw()
        plt.pause(0.001)
#-------------------------------------------------------------
def draw_obstacle_course(order, clearance):
    """
    Construct a 600x250 MatPlotLib graph with the obstacle "MM 2577" (green)
    and add a boundary of 2 extra mm around that (yellow)
    """
    # Grid
    x = np.arange(0, 600, 0.5)
    y = np.arange(0, 250, 0.5)
    X, Y = np.meshgrid(x, y)

    X_plot = X
    Y_plot = Y

    Sx = 600/180
    Sy = 250/50

    X = X/Sx
    Y = Y/Sy
    # Thickness parameter
    eps = 1

    # --- Define the First Entry M as a semi-algebraic set ---
    left_bar = (X - 5)**2 <= eps**2
    left_bar &= (Y >= 10) & (Y <= 40)

```

```python
    right_bar = (X - 25)**2 <= eps**2
    right_bar &= (Y >= 10) & (Y <= 40)

    left_diag = (Y - (-2*(X - 5) + 40))**2 <= eps**2
    left_diag &= (X >= 5) & (X <= 15)
    left_diag &= (Y >= 20) & (Y <= 40)


    right_diag = (Y - (2*(X - 25) + 40))**2 <= eps**2
    right_diag &= (X >= 15) & (X <= 25)
    right_diag &= (Y >= 20) & (Y <= 40)

    E1 = left_bar | right_bar | left_diag | right_diag

    # --- Define the Second Entry M as a semi-algebraic set ---
    left_bar2 = ((X-30) - 5)**2 <= eps**2
    left_bar2 &= (Y >= 10) & (Y <= 40)

    right_bar2 = ((X-30) - 25)**2 <= eps**2
    right_bar2 &= (Y >= 10) & (Y <= 40)

    left_diag2 = (Y - (-2*((X-30) - 5) + 40))**2 <= eps**2
    left_diag2 &= ((X-30) >= 5) & ((X-30) <= 15)
    left_diag2 &= (Y >= 20) & (Y <= 40)

    right_diag2 = (Y - (2*((X-30) - 25) + 40))**2 <= eps**2
    right_diag2 &= ((X-30) >= 15) & ((X-30) <= 25)
    right_diag2 &= (Y >= 20) & (Y <= 40)

    E2 = left_bar2 | right_bar2 | left_diag2 | right_diag2


    # --- Define the Third Entry 2 as a semi-algebraic set ---
    top = ((X-75)**2 + (Y-32)**2 >= (9-eps/2)**2)
    top &= ((X-75)**2 + (Y-32)**2 <= (9+eps/2)**2)
    top &= (X >= 66) & (X <= 84)
    top &= (Y >= 32)

    diag = (Y - (1.11*(X-84) + 32))**2 <= eps**2
    diag &= (X >= 66) & (X <= 84)
    diag &= (Y >= 12) & (Y <= 32)

    bottom = (X >= 66) & (X <= 84)
    bottom &= (Y - 12)**2 <= eps**2

    E3 = top | diag | bottom

    # --- Define the Fourth Entry 5 as a semi-algebraic set ---
    top = (X >= 93) & (X <= 108)
    top &= (Y - 40)**2 <= eps**2

    mid = (X - 93)**2 <= eps**2
    mid &= (Y >= 30) & (Y <= 40)

```

```python
    bottom = (((X-3)-93)**2 + (Y-20)**2 >= (10-eps/1.5)**2)
    bottom &= (((X-3)-93)**2 + (Y-20)**2 <= (10+eps/1.5)**2)
    bottom &= (X >= 93)
    bottom &= (Y >= 10) & (Y <= 35)

    E4 = top | mid | bottom

    # --- Define the Fifth Entry 7 as a semi-algebraic set ---
    top = (X >= 115) & (X <= 135)
    top &= (Y - 40)**2 <= eps**2

    right_diag = (Y - (2*(X - 135) + 40))**2 <= 2*eps**2
    right_diag &= (X >= 115) & (X <= 135)
    right_diag &= (Y >= 10) & (Y <= 40)

    E5 = top | right_diag

    # --- Define the Sixth Entry 7 as a semi-algebraic set ---
    top = (X >= 145) & (X <= 165)
    top &= (Y - 40)**2 <= eps**2

    right_diag = (Y - (2*(X - 165) + 40))**2 <= 2*eps**2
    right_diag &= (X >= 145) & (X <= 165)
    right_diag &= (Y >= 10) & (Y <= 40)

    E6 = top | right_diag

    # --- Define fcn to generate boundary  ---
    def get_outer_ring(mask, r=clearance):
        """
        Generate collision boundary
        """
        expanded = np.zeros_like(mask, dtype=bool)

        rows, cols = mask.shape

        for di in range(-r, r+1):
            for dj in range(-r, r+1):
                src_r0 = max(0, -di)
                src_r1 = min(rows, rows - di)
                src_c0 = max(0, -dj)
                src_c1 = min(cols, cols - dj)

                dst_r0 = max(0, di)
                dst_r1 = min(rows, rows + di)
                dst_c0 = max(0, dj)
                dst_c1 = min(cols, cols + dj)

                expanded[dst_r0:dst_r1, dst_c0:dst_c1] |= mask[src_r0:src_r1, src_c0:src_c1]

        outer_ring = expanded & (~mask)
        return outer_ring

```

```python
    # --- Generate boundaries  ---
    E1_barrier = get_outer_ring(E1, r=2*clearance)
    E2_barrier = get_outer_ring(E2, r=2*clearance)
    E3_barrier = get_outer_ring(E3, r=2*clearance)
    E4_barrier = get_outer_ring(E4, r=2*clearance)
    E5_barrier = get_outer_ring(E5, r=2*clearance)
    E6_barrier = get_outer_ring(E6, r=2*clearance)

    all_obstacles = E1 | E2 | E3 | E4 | E5 | E6
    all_barriers  = E1_barrier | E2_barrier | E3_barrier | E4_barrier | E5_barrier | E6_barrier

    all_collisions = all_obstacles | all_barriers

    # initialize the plot the first time this fcn is called,
    # but not the second time
    if order is None:
        plt.ion()
        plt.figure(figsize=(6,6))
        plt.contourf(X_plot, Y_plot, all_obstacles, levels=[0.5, 1])
        plt.contourf(X_plot, Y_plot, all_barriers, levels=[0.5, 1], colors=['yellow'])
        plt.gca().set_aspect('equal')
        plt.xlim(0, 600)
        plt.ylim(0, 250)
        plt.grid(True)
        plt.title("MM 2577 as a Semi-Algebraic Set")

    return all_collisions

#-----------------------------------------------------------
def get_inputs():
    """
    Have the user input the start and goal locations, and have them redo if if their
    entries are in collision boundaries
    """
    while True:

        # have the user input the start and goal positions
        start_str = input("Choose starting x,y,theta coordinates: ")
        goal_str = input("Choose goal x,y, theta coordinates: ")

        # split into the x and y coords
        try:
            sx, sy, st = map(int, start_str.split(","))
            gx, gy, gt = map(int, goal_str.split(","))

            start_pos = (sx, sy, st)
            goal_pos = (gx, gy, gt)

            clearance = input("Choose the clearance around the obstacles (also is the robot's radius):")
            clearance = int(clearance)

            step_size = input("Choose step size (1 to 10):")
```

```python
            step_size = int(step_size)
            # radius = input("Choose the robot's radius:")

        except:
            print("Invalid input. Use format: x,y")
            continue

        # boundary check
        if not (0 <= sx < 600 and 0 <= sy < 250):
            print("Start node outside map.")
            continue

        if not (0 <= gx < 600 and 0 <= gy < 250):
            print("Goal node outside map.")
            continue

        # check that angle (deg) is multiple of 30
        if not st % 30 == 0:
            print("Start angle is not a multiple of 30")
            continue

        if not gt % 30 == 0:
            print("Goal angle is not a multiple of 30")
            continue

        # step size check
        if step_size < 1 or step_size > 10:
            print("Invalid Step Size")
            continue

        return start_pos, goal_pos, clearance, step_size

#-----------------------------------------------------------
def check_collisions(all_collisions, start_pos, goal_pos, order):
    while True:
        sx, sy, _ = start_pos
        gx, gy, _ = goal_pos

        sx_d, sy_d = discretize([sx, sy, 0])[:2]
        gx_d, gy_d = discretize([gx, gy, 0])[:2]

        if all_collisions[sy_d, sx_d]:
            print("Start point is in collision")
            plt.close()

        elif all_collisions[gy_d, gx_d]:
            print("Goal point is in collision")
            plt.close()

        else:
            # valid → break loop
            return start_pos, goal_pos, all_collisions

        # only runs if there WAS a collision
```

```python
        start_pos, goal_pos, clearance, step_size = get_inputs()
        all_collisions = draw_obstacle_course(order, clearance)
#----------Run dijkstra---------------------
if __name__ == "__main__":


    # generate all collision spaces
    order = None

    # get user inputs
    start_pos, goal_pos, clearance, step_size = get_inputs()

    # construct obstacle course
    all_collisions = draw_obstacle_course(order, clearance)

    # check for collisions
    start_pos, goal_pos, all_collisions = check_collisions(all_collisions, start_pos, goal_pos, order)


    # run algorithm
    t_init, t_fin = run_AStar(start_pos, goal_pos, step_size, all_collisions, clearance)

    # # evaluate time for algorithm to run
    # dt = t_fin - t_init
    # print(f"The algorithm takes {dt} s to run")

    # keep plot open after completion
    plt.ioff()
    plt.show(block=True)
```